

Lab 05: Stepper Motor Control (in assembly)

ECE0202: Embedded Processors and Interfacing

Spring 2026

Stefan Zhang.....Student ID 2024141520349

Knight LiuStudent ID 2024141520361

Date Submitted: June 1, 2026

1) Pre-Lab Questions

1.1) Fill out the following timing diagrams for the GPIO outputs that create a full-stepping sequence and a half-stepping sequence.

Full stepping sequence

Signal	1	2	3	4
A / PB2	1	0	0	1
$\overline{A}$ / PB3	0	1	1	0
B / PB6	1	1	0	0
$\overline{B}$ / PB7	0	0	1	1

Half stepping sequence

Signal	1	2	3	4	5	6	7	8
A / PB2	1	1	0	0	0	0	0	1
$\overline{A}$ / PB3	0	0	0	1	1	1	0	0
B / PB6	0	1	1	1	0	0	0	0
$\overline{B}$ / PB7	0	0	0	0	0	1	1	1

1.2) How would you change your code to vary the rotation speed of the stepper motor?

The rotation speed of the stepper motor can be changed by modifying the delay between two consecutive step patterns. A smaller delay increases the stepping frequency, so the motor rotates faster. A larger delay decreases the stepping frequency, so the motor rotates more slowly.

In this program, the speed is mainly controlled by the delay value in the DELAY_MS subroutine. Increasing the delay constant reduces the motor speed, while decreasing the delay constant increases the motor speed.

1.3) How would you reverse the rotation direction of the stepper motor?

The rotation direction can be reversed by reversing the order of the stepping sequence. For example, if one direction uses

$0x0C$, $0x48$, $0xC0$, $0x84$,

then the opposite direction uses

$0x84$, $0xC0$, $0x48$, $0x0C$.

In the assembly program, this is implemented by increasing or decreasing the step index before reading the next value from the step table.

2) Experimental Setup

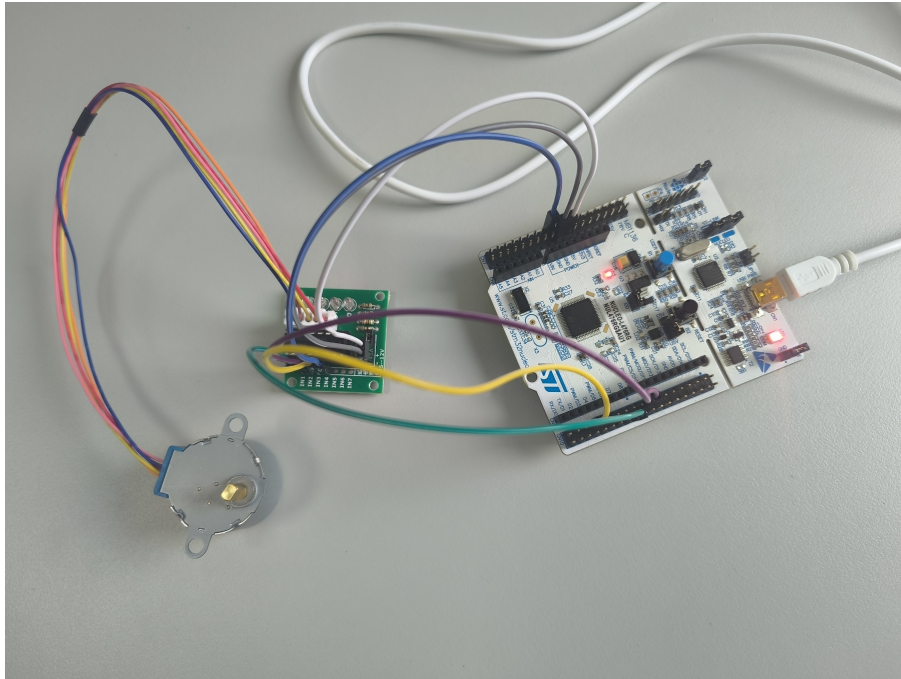


Figure 2: Hardware connection of the stepper motor circuit

3) Code

The main assembly program used in this experiment is shown below.

```
***** (C) Yifeng ZHU *****
; @file    main.s
; @author  Yifeng Zhu
; @date    May-17-2015
*****

INCLUDE core_cm4_constants.s
INCLUDE stm32l476xx_constants.s

IMPORT      System_Clock_Init

PRESERVE8

AREA      main, CODE, READONLY
EXPORT    __main
ENTRY

__main    PROC

    BL      System_Clock_Init

    LDR      R0, =RCC_BASE
    LDR      R1, [R0, #RCC_AHB2ENR]
    ORR      R1, R1, #RCC_AHB2ENR_GPIOBEN
    ORR      R1, R1, #RCC_AHB2ENR_GPIOCEN
    STR      R1, [R0, #RCC_AHB2ENR]

    LDR      R0, =GPIOC_BASE
    LDR      R1, [R0, #GPIO_MODER]
```

```

    BIC      R1, R1, #(3 << (13 * 2))
    STR      R1, [R0, #GPIO_MODER]

    LDR      R0, =GPIOB_BASE
    LDR      R1, [R0, #GPIO_MODER]

    BIC      R1, R1, #(3 << (2 * 2))
    BIC      R1, R1, #(3 << (3 * 2))
    BIC      R1, R1, #(3 << (6 * 2))
    BIC      R1, R1, #(3 << (7 * 2))

    ORR      R1, R1, #(1 << (2 * 2))
    ORR      R1, R1, #(1 << (3 * 2))
    ORR      R1, R1, #(1 << (6 * 2))
    ORR      R1, R1, #(1 << (7 * 2))

    STR      R1, [R0, #GPIO_MODER]

    LDR      R1, [R0, #GPIO_OTYPER]
    BIC      R1, R1, #0xCC
    STR      R1, [R0, #GPIO_OTYPER]

    BL       MOTOR_OFF

    MOV      R4, #0

MAIN_LOOP

    LDR      R0, =GPIOC_BASE
    LDR      R1, [R0, #GPIO_IDR]
    ANDS     R1, R1, #(1 << 13)
    BNE      MAIN_LOOP

WIPER_ROUTINE

    MOV      R4, #0
    LDR      R5, =825

CCW_LOOP
    BL       STEP_CCW
    BL       DELAY_MS
    SUBS     R5, R5, #1
    BNE      CCW_LOOP

    MOV      R6, #50

CCW_PAUSE
    BL       DELAY_MS
    SUBS     R6, R6, #1
    BNE      CCW_PAUSE

    MOV      R4, #0
    LDR      R5, =825

CW_LOOP
    BL       STEP_CW
    BL       DELAY_MS
    SUBS     R5, R5, #1
    BNE      CW_LOOP

    BL       MOTOR_OFF

```

```

B      MAIN_LOOP

STEP_CCW
    PUSH    {LR}

    ADD     R4, R4, #1
    CMP     R4, #4
    BNE     WRITE_CCW
    MOV     R4, #0

WRITE_CCW
    BL      DRIVE_MOTOR

    POP     {PC}

STEP_CW
    PUSH    {LR}

    SUBS    R4, R4, #1
    BPL     WRITE_CW
    MOV     R4, #3

WRITE_CW
    BL      DRIVE_MOTOR

    POP     {PC}

DRIVE_MOTOR
    PUSH    {R0-R3, LR}

    LDR     R0, =GPIOB_BASE

    LDR     R1, [R0, #GPIO_ODR]

    BIC     R1, R1, #0xCC

    LDR     R2, =STEP_TABLE
    LDRB    R3, [R2, R4]

    ORR     R1, R1, R3
    STR     R1, [R0, #GPIO_ODR]

    POP     {R0-R3, PC}

MOTOR_OFF
    PUSH    {R0-R1, LR}

    LDR     R0, =GPIOB_BASE
    LDR     R1, [R0, #GPIO_ODR]
    BIC     R1, R1, #0xCC
    STR     R1, [R0, #GPIO_ODR]

    POP     {R0-R1, PC}

DELAY_MS
    PUSH    {R0}

    LDR     R0, =150000

DELAY_LOOP
    SUBS    R0, R0, #1
    BNE     DELAY_LOOP

```

```

POP      {R0}
BX       LR

ENDP

ALIGN

AREA myData, DATA, READWRITE
ALIGN

STEP_TABLE
DCB      0x0C
DCB      0x48
DCB      0xC0
DCB      0x84

END

```

4) Participation and Contribution

Table 1: Participation and contribution of group members

Name	Participation and Contribution
Stefan Zhang	Code programming; testing; report writing
Knight Liu	Hardware connection; circuit testing